

BandUp

Thiết kế hệ thống

System Design Specification

Phiên bản: 1.0

Trạng thái: Baseline kỹ thuật Web MVP

Kiến trúc: Modular Monolith

Triển khai: VPS + Docker Compose

Background jobs: Hangfire

Ngày chốt: 18/07/2026

Tài liệu làm căn cứ cho thiết kế module, database, API, background processing, security, deployment và vận hành hệ thống BandUp

Kiểm soát tài liệu

Thuộc tính	Giá trị
Tên tài liệu	04_System_Design
Phiên bản	1.0
Trạng thái	Baseline kỹ thuật Web MVP
Ngày chốt	18/07/2026
Ngôn ngữ	Tiếng Việt
Sản phẩm	BandUp
Tài liệu đầu vào	01_BandUp_Overview; 02_Product_Requirements; 03_UI_UX_Design
Tài liệu đầu ra liên quan	Database Design; API Contract; Testing Plan; Deployment Runbook; ADR; Sprint Backlog

Giới hạn tài liệu. Tài liệu này mô tả kiến trúc, module, luồng xử lý, trạng thái, dữ liệu, bảo mật, background jobs, triển khai và vận hành. Nó không thay thế tài liệu UI/UX, test case chi tiết, sprint plan hay source code hoàn chỉnh.

Mục lục

Kiểm soát tài liệu	1
Mục đích tài liệu	6
1 Các quyết định kiến trúc đã chốt	6
2 Mục tiêu và thuộc tính chất lượng kiến trúc	7
2.1 Mục tiêu	7
2.2 Thuộc tính chất lượng ưu tiên	7
3 Lựa chọn Modular Monolith	8
3.1 Lý do	8
3.2 Modular Monolith không phải monolith tùy tiện	8
3.3 Điều kiện cân nhắc microservice sau này	8
4 System Context	8
4.1 Actor và hệ thống ngoài	8
5 Kiến trúc runtime	9
5.1 Các tiến trình chính	9
5.2 Tách API và Worker	10

6 Topology triển khai VPS	10
6.1 Same-origin	10
6.2 Hangfire Dashboard	10
7 Công nghệ frontend	11
7.1 Stack	11
7.2 Nguyên tắc state	11
8 Công nghệ backend	11
8.1 Stack	11
8.2 Backend là nguồn quyết định nghiệp vụ	11
9 Database và persistence	11
9.1 PostgreSQL cho development và production	11
9.2 EF Core thay vì ISqlProvider	12
9.3 Logical schema	12
9.4 Transaction quan trọng	12
10 Các module nghiệp vụ	12
11 Dependency và ranh giới module	13
11.1 Quy tắc	14
12 Cấu trúc source code đề xuất	14
13 Authentication: Google Login và cookie-session	14
13.1 Luồng đăng nhập	15
13.2 Vì sao vẫn cần cookie	15
13.3 Cookie baseline	15
14 Authorization: role, permission và policy	15
14.1 Mô hình	15
14.2 Role MVP	16
14.3 Permission nhạy cảm	16
15 Browser security: CSRF, CORS và XSS	16
16 Draft và Autosave	16
16.1 Luồng	17
16.2 Optimistic concurrency	17
16.3 Local recovery	17

17 Submission State Machine	17
18 Submit bài và reserve credit	18
18.1 Luồng transaction	18
19 Credit Ledger và Reservation	18
19.1 Thành phần	18
19.2 Transaction type	18
19.3 State	18
19.4 Ràng buộc	19
20 AI Grading Pipeline	19
20.1 Một model chấm chính	19
20.2 Luồng worker	19
20.3 Structured output validation	19
21 Hangfire Background Jobs	20
21.1 Kết luận lựa chọn	20
21.2 Vai trò của Hangfire	20
21.3 Production topology	20
21.4 Queue	20
21.5 Concurrency baseline	20
21.6 Job argument	20
21.7 Automatic retry	20
21.8 Idempotency của job	21
21.9 Recovery job	21
21.10 Storage provider risk	21
22 AI Retry, Circuit Breaker và Kill Switch	21
22.1 Retry policy	21
22.2 Circuit breaker	22
22.3 Kill switch	22
23 Prompt Versioning	22
24 Third-party abstraction	22
24.1 Interface nên có	22
24.2 Không abstraction quá mức	22
24.3 Implementation baseline	22

25 payOS Payment Flow	23
25.1 Nguồn sự thật	23
25.2 Payment state	23
25.3 Webhook idempotency	23
26 Support, grading dispute và refund	23
26.1 Auto release	24
27 Progress, Mistake Bank và Redo	24
28 Object Storage	24
29 Notification và email	24
30 API Design	24
30.1 Route convention	24
30.2 Problem Details	24
30.3 Pagination và filter	25
30.4 Idempotency key	25
31 Database entities cấp cao	25
31.1 Identity	25
31.2 Writing và grading	25
31.3 Learning	25
31.4 Billing và support	25
31.5 Operations	25
32 Configuration và secret management	26
32.1 Environment secret	26
32.2 Business settings	26
33 Logging, Audit và Observability	26
33.1 Structured log fields	26
33.2 Audit	26
33.3 Health check	26
33.4 Metric	26
34 Rate Limiting và abuse control	26
35 Data Retention	27

36 Account Deletion	27
37 Backup và Disaster Recovery	28
37.1 Backup	28
37.2 Mục tiêu tham khảo	28
38 Failure Scenarios	28
39 Security Threat Model cơ bản	28
40 Attachment Security	29
41 Scalability Roadmap	29
41.1 Giai đoạn 1	29
41.2 Giai đoạn 2	29
41.3 Giai đoạn 3	29
42 Hướng mở rộng Mobile App	29
43 Testing Boundary	29
44 Migration, Seed và Admin Bootstrap	30
45 State Machine cần đặc tả chi tiết	30
46 Sequence Diagram cần bổ sung	30
47 Architectural Decision Records	31
48 Rủi ro kỹ thuật và biện pháp	31
49 Implementation Checklist	32
49.1 P0 - Foundation	32
49.2 P0 - Core writing/grading	32
49.3 P1 - Product differentiation	32
49.4 P1 - Production hardening	32
50 Tiêu chí chấp nhận System Design	32
51 Kết luận	33
Nguồn tham khảo	34

Mục đích tài liệu

Tài liệu này chuyển các yêu cầu sản phẩm và UI/UX của BandUp thành một baseline kỹ thuật đủ rõ để bắt đầu triển khai. Mục tiêu là giảm quyết định ngẫu hứng trong khi code, giữ ranh giới module, kiểm soát chi phí AI và bảo đảm các nghiệp vụ nhạy cảm như credit, thanh toán, quyền admin và xóa dữ liệu có thiết kế nhất quán.

Tài liệu trả lời các câu hỏi:

- Hệ thống BandUp gồm những tiến trình và module nào?
- Google Login, cookie-session, role và permission phối hợp ra sao?
- Submission, credit và AI grading được xử lý thế nào để không trùng và không mất quyền lợi user?
- Hangfire có đủ dùng không, cần cấu hình queue, worker, retry và recovery như thế nào?
- payOS webhook được xác minh và cộng credit thế nào?
- Dữ liệu được lưu, backup, xóa và anonymize ra sao?
- Hệ thống được deploy trên VPS và mở rộng trong tương lai như thế nào?

1. Các quyết định kiến trúc đã chốt

Hạng mục	Phương án baseline	Ghi chú
Kiến trúc ứng dụng	Modular Monolith	Một codebase và một hệ thống deploy chính, nhưng module có ranh giới rõ.
Nền tảng sản phẩm	Desktop Web-first	Mobile app không thuộc MVP; backend API vẫn được thiết kế để mở rộng sau.
Frontend	React + TypeScript + Vite	SPA gọi ASP.NET Core API.
Backend	ASP.NET Core Web API	Business rules nằm ở backend.
Database	PostgreSQL + EF Core/Npgsql	Dùng cùng provider ở development và production.
Authentication	Google OIDC + application cookie	Google xác minh danh tính; BandUp quản lý session, role và account status.
Authorization	Role + Permission + Policy	Một user có nhiều role; một role có nhiều user.
Background jobs	Hangfire	API và worker tách process/container ở production.
AI	OpenAI qua interface	Một model chính được cấu hình và benchmark; structured output bắt buộc.

Hạng mục	Phương án baseline	Ghi chú
Thanh toán	payOS	Cộng credit dựa trên webhook đã xác minh và idempotency.
Credit	Ledger + Reservation	Reserve khi submit; consume khi thành công; release khi thất bại.
Trạng thái chấm	Polling trong MVP	Chưa cần SignalR.
Deploy	VPS + Docker Compose + Caddy	Same-origin deployment được ưu tiên.
Object storage	Local ở dev; S3-compatible ở production	Provider cụ thể chốt trong Deployment Plan.
Cache/Redis	Chưa bắt buộc	Chỉ thêm khi có số liệu chứng minh nhu cầu.

Nguyên tắc tổng quát: BandUp ưu tiên một hệ thống đủ đơn giản để solo developer hoặc team nhỏ triển khai, nhưng các điểm khó thay đổi sau này — identity, authorization, credit ledger, payment idempotency, prompt versioning và data deletion — phải được thiết kế đúng từ đầu.

2. Mục tiêu và thuộc tính chất lượng kiến trúc

2.1 Mục tiêu

1. Một developer hoặc nhóm nhỏ có thể hiểu, debug, deploy và vận hành.
2. HTTP request không phải chờ AI chấm xong.
3. Mọi thay đổi credit và payment có ledger, transaction và audit.
4. Dịch vụ ngoài có abstraction tại biên hệ thống, không tràn vào domain.
5. Có thể phục vụ MVP, beta và early production trên VPS.
6. Có thể scale worker và API riêng mà chưa cần microservice.
7. Dữ liệu user có ownership, retention và deletion rõ ràng.

2.2 Thuộc tính chất lượng ưu tiên

Thuộc tính	Mục tiêu	Cách hỗ trợ trong thiết kế
Maintainability	Thay đổi một module không phá toàn hệ thống	Module ownership, DTO, service contract, migration và ADR.
Reliability	Không mất bài, credit hoặc payment	Autosave, transaction, ledger, idempotency, retry và recovery job.

Thuộc tính	Mục tiêu	Cách hỗ trợ trong thiết kế
Security	Bảo vệ session, dữ liệu và quyền admin	Cookie bảo mật, CSRF, ownership, policy, audit và least privilege.
Cost control	Không phát sinh AI cost ngoài kiểm soát	Queue, concurrency limit, quota, reserve credit, budget cap và kill switch.
Scalability	Tăng worker/API khi cần	API stateless tương đối, worker riêng, shared DB/storage, queue rõ.
Observability	Điều tra được lỗi và chi phí	Structured log, trace id, AI usage, payment event và dashboard.
Portability	Có thể đổi VPS/provider	Docker, environment configuration và third-party interfaces.

3. Lựa chọn Modular Monolith

3.1 Lý do

BandUp chưa cần microservice vì team ban đầu nhỏ, nghiệp vụ có nhiều transaction chéo và tải chưa chứng minh nhu cầu scale độc lập. Submission liên quan chặt đến credit, grading, progress và notification; tách thành dịch vụ phân tán quá sớm sẽ làm tăng độ phức tạp về network, transaction, retry, tracing, deployment và security.

3.2 Modular Monolith không phải monolith tùy tiện

Một database vật lý không có nghĩa module được tự do sửa bảng của nhau. Mỗi module sở hữu dữ liệu và application service của mình. Module khác tương tác qua contract hoặc service, không dùng SQL trực tiếp để bỏ qua business rules.

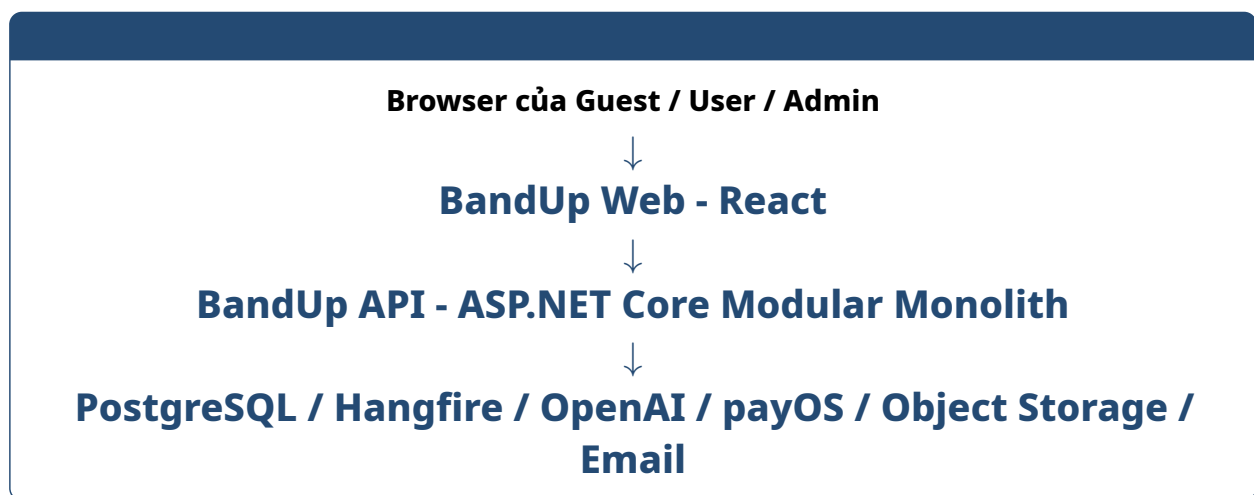
3.3 Điều kiện cân nhắc microservice sau này

- Một module cần scale khác biệt rõ và đã có số liệu.
- Team đủ lớn để sở hữu dịch vụ độc lập.
- Release chung trở thành nút thắt đáng kể.
- Ranh giới module đã ổn định trong monolith.
- Chi phí vận hành distributed system có lợi hơn chi phí hiện tại.

4. System Context

4.1 Actor và hệ thống ngoài

Actor/Hệ thống	Vai trò
Guest	Xem landing page, pricing, policy và bắt đầu Google Login.
User	Viết bài, chấm, xem tiến bộ, mua credit và yêu cầu hỗ trợ.
Admin	Quản lý theo permission trong một Admin Portal.
Google Identity	Xác minh danh tính qua OIDC/OAuth; không quản lý role BandUp.
OpenAI	Tạo structured grading result từ essay và prompt version.
payOS	Tạo payment link/VietQR và gửi webhook thanh toán.
Email provider	Gửi email hỗ trợ, thanh toán và bảo mật khi được bật.
Object storage	Lưu avatar, support attachment và file export.
Monitoring/Backup	Theo dõi health, log, metric và lưu backup ngoài VPS.



5. Kiến trúc runtime

5.1 Các tiến trình chính

Tiến trình	Trách nhiệm	Không chịu trách nhiệm
BandUp Web	UI, navigation, editor state, polling, client validation	Quyền, quota, credit, payment state và grading decisions.
BandUp API	Auth, authorization, business logic, database transaction, enqueue job	Chạy AI dài trong request.
BandUp Worker	Hangfire server, grading, email, cleanup, aggregation, deletion	Nhận request trực tiếp từ public browser.
PostgreSQL	Dữ liệu nghiệp vụ và persistent Hangfire storage	Object file lớn và secret.
Reverse Proxy	TLS, routing, headers, static files, rate limit biên cơ bản	Business authorization.

5.2 Tách API và Worker

Development có thể chạy Hangfire Server cùng API để giảm số process. Production phải tách Worker thành container riêng. Việc này cho phép restart API mà không dừng job, giới hạn tài nguyên AI riêng, tăng worker độc lập và tránh latency của background work ảnh hưởng request web.

6. Topology triển khai VPS

```

Internet
|
v
Caddy (HTTPS, reverse proxy)
|-- /          -> bandup-web
|-- /api/*      -> bandup-api
`-- /hangfire   -> bandup-worker dashboard (protected)

Docker Compose
|-- bandup-web
|-- bandup-api
|-- bandup-worker
|-- postgres
`-- caddy

External
|-- OpenAI
|-- payOS
|-- S3-compatible storage
`-- Email provider
  
```

6.1 Same-origin

Frontend và API ưu tiên cùng origin:

```

https://bandup.vn
https://bandup.vn/api/v1/...
https://bandup.vn/admin
  
```

Lợi ích gồm cookie đơn giản hơn, giảm CORS, CSRF rõ ràng và dễ cấu hình Google callback.

6.2 Hangfire Dashboard

Hangfire Dashboard không được mở public. Chỉ permission vận hành phù hợp được truy cập; nên có xác thực, giới hạn IP nếu khả thi và không đưa essay/secret vào job arguments.

7. Công nghệ frontend

7.1 Stack

- React + TypeScript + Vite.
- React Router cho route.
- TanStack Query cho server state, cache, polling và pagination.
- React Hook Form cùng schema validation phù hợp.
- Editor là textarea/editor text đơn giản; không cần rich text.

7.2 Nguyên tắc state

- Server là nguồn dữ liệu chính cho draft, submission, credit và result.
- Local state chỉ dùng cho modal, tab, filter và bản recovery tạm.
- Không tin role, balance, paymentStatus hoặc submissionStatus do frontend gửi lên.
- API response dùng DTO, không phụ thuộc cấu trúc database.

8. Công nghệ backend

8.1 Stack

- ASP.NET Core Web API.
- Entity Framework Core + Npgsql.
- ASP.NET Core Identity hoặc identity layer tương đương cho local user, external login và security stamp.
- Google OpenID Connect.
- Cookie Authentication.
- Hangfire + PostgreSQL storage provider đã được integration test.
- Structured logging.
- OpenAPI/Swagger cho development và contract review.

8.2 Backend là nguồn quyết định nghiệp vụ

Backend quyết định ownership, permission, essay limit, quota, credit, AI settings, payment state, prompt publication, retention và deletion. Frontend chỉ phản ánh quyết định này.

9. Database và persistence

9.1 PostgreSQL cho development và production

Dùng cùng provider giúp tránh khác biệt migration, kiểu dữ liệu, JSON, case sensitivity và query giữa môi trường. Development chạy PostgreSQL bằng Docker; production có thể chạy

container riêng trên VPS giai đoạn đầu hoặc chuyển managed database sau.

9.2 EF Core thay vì ISqlProvider

Không tạo một interface database khổng lồ. EF Core DbContext và repository/application service theo module là lớp truy cập dữ liệu. Nếu có truy vấn Dapper đặc biệt, dùng IDbConnectionFactory nhỏ và có mục đích rõ.

9.3 Logical schema

Có thể phân tách bằng PostgreSQL schema:

```
identity.*
writing.*
grading.*
learning.*
billing.*
support.*
security.*
operations.*
```

Giai đoạn đầu có thể dùng một schema nếu migration đơn giản hơn, nhưng naming và ownership module phải giữ nguyên.

9.4 Transaction quan trọng

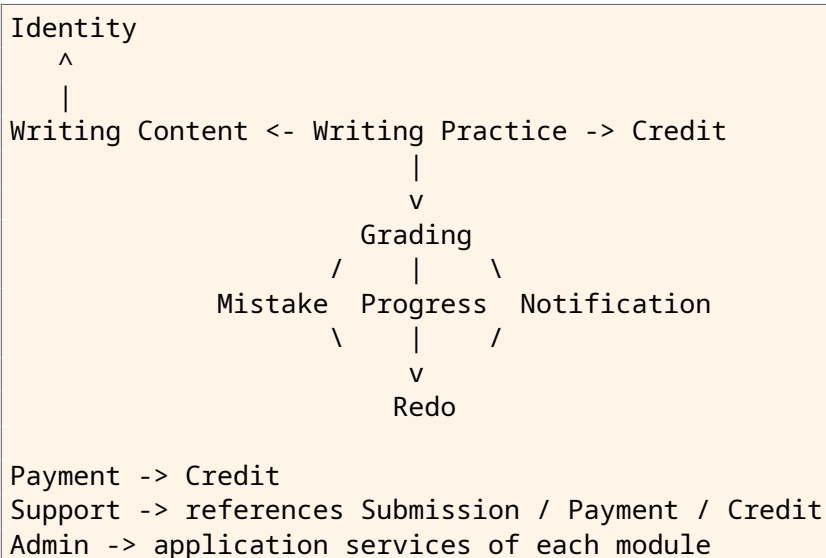
- Tạo submission + reserve credit.
- Lưu assessment + consume credit + complete submission.
- Fail grading + release credit.
- Payment paid + grant credit.
- Admin credit adjustment.
- Account deletion/anonymization.

10. Các module nghiệp vụ

Module	Trách nhiệm chính	Dữ liệu sở hữu điển hình
Identity	Google Login, cookie, session, role, permission, account status	Users, ExternalLogins, Sessions, Roles, Permissions.
User Profile	Onboarding và thông tin học tập	UserProfiles, Preferences.
Writing Content	Prompt, tag, model essay, publish workflow	WritingPrompts, PromptTags, ModelEssays.
Writing Practice	Draft, autosave, timer, submission và history	WritingDrafts, WritingSubmissions.

Module	Trách nhiệm chính	Dữ liệu sở hữu điển hình
Grading	Prompt version, attempts, OpenAI, assessment, usage	GradingAttempts, Assessments, Criteria, AiModelCalls.
Mistake Bank	Taxonomy và lỗi từng bài	ErrorTaxonomies, ErrorOccurrences, UserMistakeStats.
Progress	Trend, aggregate và insight	ProgressSnapshots, derived metrics.
Redo	Recommendation và comparison	RedoRecommendations, RedoAttempts.
Credit	Wallet, reservation và ledger	CreditWallets, Transactions, Reservations.
Payment	Package, payOS order, webhook, refund	PaymentOrders, WebhookEvents, Refunds.
Support	Ticket, message, attachment và dispute	SupportTickets, Messages, Attachments.
Notification	In-app và email event	Notifications, DeliveryAttempts.
Admin	Dashboard và orchestration theo permission	Không sở hữu dữ liệu module khác.
Security	Abuse, block, rate-limit, kill switch, audit	SecurityEvents, BlockedClients, AuditLogs.
Analytics	Product events và funnel không chứa essay text	ProductEvents, DailyMetrics.

11. Dependency và ranh giới module



11.1 Quy tắc

- Grading không tự cấp hoặc trừ credit ngoài Credit Service.
- Payment không sửa wallet trực tiếp.
- Admin không update bảng module khác bằng SQL.
- Progress không chỉnh Assessment.
- Analytics chỉ nhận event, không điều khiển transaction nghiệp vụ.
- Shared Kernel chỉ chứa primitive và abstraction thực sự dùng chung; không trở thành nơi chứa mọi thứ.

12. Cấu trúc source code đề xuất

```
src/  
|-- BandUp.Api/  
|-- BandUp.Worker/  
|-- BandUp.Contracts/  
|-- BandUp.SharedKernel/  
|-- BandUp.Infrastructure/  
|   |-- Persistence/  
|   |-- OpenAI/  
|   |-- PayOS/  
|   |-- Storage/  
|   |-- Email/  
|   |-- Observability/  
|-- BandUp.Modules/  
|   |-- Identity/  
|   |-- UserProfiles/  
|   |-- WritingContent/  
|   |-- WritingPractice/  
|   |-- Grading/  
|   |-- MistakeBank/  
|   |-- Progress/  
|   |-- Redo/  
|   |-- Credit/  
|   |-- Payment/  
|   |-- Support/  
|   |-- Notification/  
|   |-- Admin/  
|   |-- Security/  
|-- Analytics/
```

Có thể bắt đầu với ít project hơn và chia folder theo module. Chất lượng dependency quan trọng hơn số lượng project.

13. Authentication: Google Login và cookie-session

13.1 Luồng đăng nhập



13.2 Vì sao vẫn cần cookie

Google chỉ xác minh danh tính ở bước login. Cookie của BandUp giúp xác định user cho các request sau và gắn local role, permission, session, credit, account status. BandUp không lưu mật khẩu Google.

13.3 Cookie baseline

Thuộc tính	Cấu hình
HttpOnly	true.
Secure	true ở production.
SameSite	Lax, kiểm thử Google callback và payment return flow.
Session duration	8–24 giờ theo setting.
Remember me	Tùy chọn riêng, thời hạn dài hơn và có revoke.
Data Protection keys	Persist ra volume/shared store; không để mất khi container restart.
CSRF	Bắt buộc với request thay đổi dữ liệu.

14. Authorization: role, permission và policy

14.1 Mô hình

Users ↔ UserRoles ↔ Roles ↔ RolePermissions ↔ Permissions

14.2 Role MVP

- USER.
- CONTENT_ADMIN.
- USER_SUPPORT_ADMIN.
- FINANCE_ADMIN.
- SECURITY_ADMIN.
- SUPER_ADMIN.

14.3 Permission nhạy cảm

Permission	Ý nghĩa
Prompt.Publish	Publish/unpublish đề và bài mẫu.
User.Suspend	Khóa user, yêu cầu reason và audit.
User.ViewEssayForSupport	Xem essay user vì mục đích hỗ trợ; permission riêng.
Credit.Adjust	Cộng/trừ credit qua ledger, không sửa balance trực tiếp.
Payment.Refund	Ghi nhận và xử lý refund.
Security.BlockClient	Block user/IP/client.
System.ToggleAi	Bật/tắt AI grading.
System.Maintenance	Bật maintenance mode.
Admin.ManageRoles	Gán role admin; chỉ Super Admin.

15. Browser security: CSRF, CORS và XSS

- Same-origin là baseline; không dùng AllowAnyOrigin cùng credentials.
- Request POST/PUT/PATCH/DELETE dùng anti-forgery token hoặc cơ chế CSRF tương đương.
- Cookie không được JavaScript đọc.
- AI output và user content được encode; không render raw HTML.
- Cấu hình Content Security Policy, X-Content-Type-Options và frame policy.
- Endpoint luôn kiểm tra ownership, tránh IDOR khi đổi ID trên URL.

16. Draft và Autosave

16.1 Luồng



16.2 Optimistic concurrency

Draft có Version hoặc concurrency token. Nếu tab cũ gửi version lỗi thời, backend trả 409 Conflict; UI cho user chọn tải bản server hoặc sao chép nội dung local. Không âm thầm ghi đè.

16.3 Local recovery

Browser có thể giữ bản recovery tạm khi mất mạng. UI phải phân biệt “Đã lưu trên server” và “Chưa đồng bộ”. Khi user logout hoặc xóa tài khoản, local recovery tương ứng phải được xóa.

17. Submission State Machine

Trạng thái	Chuyển tiếp hợp lệ	Ý nghĩa
DRAFT	SUBMITTED	Nội dung còn sửa được.
SUBMITTED	QUEUED	Validation và credit reservation thành công.
QUEUED	PROCESSING, CANCELLED	Đang chờ worker; có thể hủy nếu chưa bắt đầu và policy cho phép.
PROCESSING	COMPLETED, RETRY_SCHEDULED, FAILED	Worker đang xử lý.
RETRY_SCHEDULED	PROCESSING, FAILED	Chờ attempt tiếp theo.
COMPLETED	Terminal	Assessment đã lưu, credit đã consume.
FAILED	Regrade request mới	Credit đã release; bài được giữ để thử lại.

Trạng thái	Chuyển tiếp hợp lệ	Ý nghĩa
CANCELLED	Terminal hoặc submit lại	Credit release nếu đã reserve.

18. Submit bài và reserve credit

18.1 Luồng transaction

1. Kiểm tra authentication và ownership draft.
2. Kiểm tra account status, word count 200/250/350 và idempotency key.
3. Kiểm tra rate limit, AI settings, daily budget và credit.
4. Tạo immutable WritingSubmission.
5. Tạo CreditReservation và ledger entry RESERVE.
6. Tạo record cần enqueue/outbox marker.
7. Commit transaction.
8. Enqueue Hangfire job; nếu thất bại, recovery job sẽ phát hiện và enqueue lại.

Không được trừ hoặc reserve credit chỉ ở frontend. Không được tạo submission trước rồi reserve credit ở transaction khác vì hai thao tác có thể lệch trạng thái.

19. Credit Ledger và Reservation

19.1 Thành phần

- CreditWallet: số dư tính nhanh và trạng thái wallet.
- CreditTransaction: ledger bất biến cho mọi thay đổi.
- CreditReservation: credit đang giữ cho một submission.

19.2 Transaction type

PURCHASE, BONUS, RESERVE, CONSUME, RELEASE, REFUND, ADMIN_ADJUSTMENT, EXPIRATION

19.3 State



thất bại/hủy

19.4 Ràng buộc

- Một submission có tối đa một reservation active.
- Consume/release là idempotent.
- Không sửa lịch sử transaction; sai phải tạo adjustment mới.
- Balance có thể rebuild từ ledger để kiểm tra reconciliation.

20. AI Grading Pipeline

20.1 Một model chấm chính

MVP dùng một model OpenAI được cấu hình trong `AiSettings.PrimaryGradingModel`. Model ID không hard-code trong domain; trước public production phải benchmark trên tập bài có điểm tham chiếu. Không xây chuỗi nhiều model trong MVP.

20.2 Luồng worker

1. Load submission theo ID.
2. Dừng nếu đã COMPLETED hoặc đã có assessment thành công.
3. Kiểm tra credit reservation còn hợp lệ.
4. Chuyển PROCESSING và tạo GradingAttempt.
5. Load active GradingPromptVersion và taxonomy.
6. Build request, gọi `IAiGradingProvider`.
7. Parse/validate structured output.
8. Trong transaction: lưu assessment, criteria, corrections, suggestions, errors; consume credit; complete submission; tạo notification.
9. Ghi AI usage và cost estimate.

20.3 Structured output validation

Backend kiểm tra:

- Overall và bốn criteria từ 0 đến 9, bước 0.5.
- Đủ field bắt buộc.
- Error code thuộc taxonomy hoặc map OTHER.
- Giới hạn số item và độ dài chuỗi.
- Không có HTML/script nguy hiểm.
- Không tin raw output làm dữ liệu hiển thị trực tiếp.

21. Hangfire Background Jobs

21.1 Kết luận lựa chọn

Hangfire đủ dùng cho MVP và early production của BandUp. Nó phù hợp với grading, notification, payment reconciliation, progress aggregation, retention cleanup và data deletion. Hệ thống chưa cần RabbitMQ, Kafka hay cloud message bus.

21.2 Vai trò của Hangfire

Hangfire chịu trách nhiệm persist job, phân phối cho worker, quản lý queue, lịch recurring và trạng thái job. Hangfire không tự bảo đảm business idempotency, credit consistency hoặc payment correctness; các phần đó thuộc module nghiệp vụ.

21.3 Production topology

- bandup-api: enqueue, không chạy grading worker.
- bandup-worker: chạy Hangfire Server.
- Dùng shared PostgreSQL storage.
- Worker có thể scale thành nhiều instance sau này.

21.4 Queue

Queue	Job	Ưu tiên/tài nguyên
grading	Gọi OpenAI và lưu assessment	Concurrency thấp, kiểm soát cost.
payments	Reconciliation, webhook recovery	Ưu tiên cao vì liên quan tiền.
notifications	In-app/email delivery	Có thể retry độc lập.
maintenance	Cleanup, aggregate, retention, deletion	Thấp, chạy theo lịch.

21.5 Concurrency baseline

Bắt đầu `MaxConcurrentAiJobs = 3`, cấu hình được. Không dùng worker count mặc định để gọi AI không giới hạn. Tăng concurrency dựa trên rate limit, latency, cost, queue time và tài nguyên thực tế.

21.6 Job argument

Job chỉ nhận ID nhỏ như `SubmissionId`, `PaymentOrderId` hoặc `DeletionRequestId`. Không serialize essay, API key, cookie hoặc payload nhạy cảm vào Hangfire storage.

21.7 Automatic retry

Hangfire có retry mặc định; grading job phải tắt hoặc giảm retry framework và để Grading Service phân loại lỗi. Nếu cả Hangfire và service cùng retry, số AI call có thể nhân lên ngoài

ý muốn.

21.8 Idempotency của job

Trước mỗi side effect:

- Kiểm tra state hiện tại.
- Dùng unique constraint khi phù hợp.
- Consume/release credit idempotent.
- Webhook và payment grant có processed marker.
- Assessment có uniqueness theo successful logical grading.

21.9 Recovery job

Recurring recovery tìm:

- Submission QUEUED quá ngưỡng nhưng chưa có job/enqueued marker.
- Reservation quá lâu do worker crash.
- Payment PAID nhưng credit chưa grant.
- Deletion request bị treo.

Recovery không tạo duplicate nhờ idempotency key và state check.

21.10 Storage provider risk

Hangfire.PostgreSql là provider cộng đồng, nên cần integration test cho enqueue, retry, distributed locking, restart worker và nhiều worker. Nếu provider trở thành rủi ro lớn, có thể dùng Hangfire SQL Server storage hoặc thay job system mà không đổi domain contract IGradingJobQueue.

22. AI Retry, Circuit Breaker và Kill Switch

22.1 Retry policy

Tối đa ba lần gọi AI tổng cộng cho một logical grading:

Attempt	Delay gợi ý	Ghi chú
1	Ngay lập tức	Gọi chính.
2	Khoảng 30 giây + jitter	Timeout, 429, 5xx hoặc output có thể sửa.
3	Khoảng 2 phút + jitter	Attempt cuối; hết lỗi thì fail và release credit.

Không retry lỗi 400 do request/schema, 401/403, essay invalid, budget exceeded hoặc AI disabled.

22.2 Circuit breaker

Nếu provider lỗi liên tiếp vượt ngưỡng:

- Mở circuit, không gọi mới trong thời gian cooldown.
- Reschedule job thay vì fail ngay.
- Credit giữ ở RESERVED trong thời gian gián đoạn có giới hạn.
- Cảnh báo admin và hiển thị status thân thiện.
- Half-open bằng một số request thử trước khi mở lại.

22.3 Kill switch

AiGradingEnabled, FreeGradingEnabled, MaintenanceMode, DailyAiBudgetLimit, MaxConcurrentAiJobs là business settings. Thay đổi cần permission, reason và audit.

23. Prompt Versioning

Không ghi đè prompt đang active. Mỗi version lưu system prompt, template, output schema, model config, status và activation time. Assessment lưu PromptVersionId cùng ModelName, nhờ đó có thể so sánh chất lượng và điều tra lịch sử.

24. Third-party abstraction

24.1 Interface nên có

```
IAiGradingProvider
IPaymentGateway
IObjectStorage
ISender
IGradingJobQueue
IClock (khi business rule ược thực ở thời gian)
```

24.2 Không abstraction quá mức

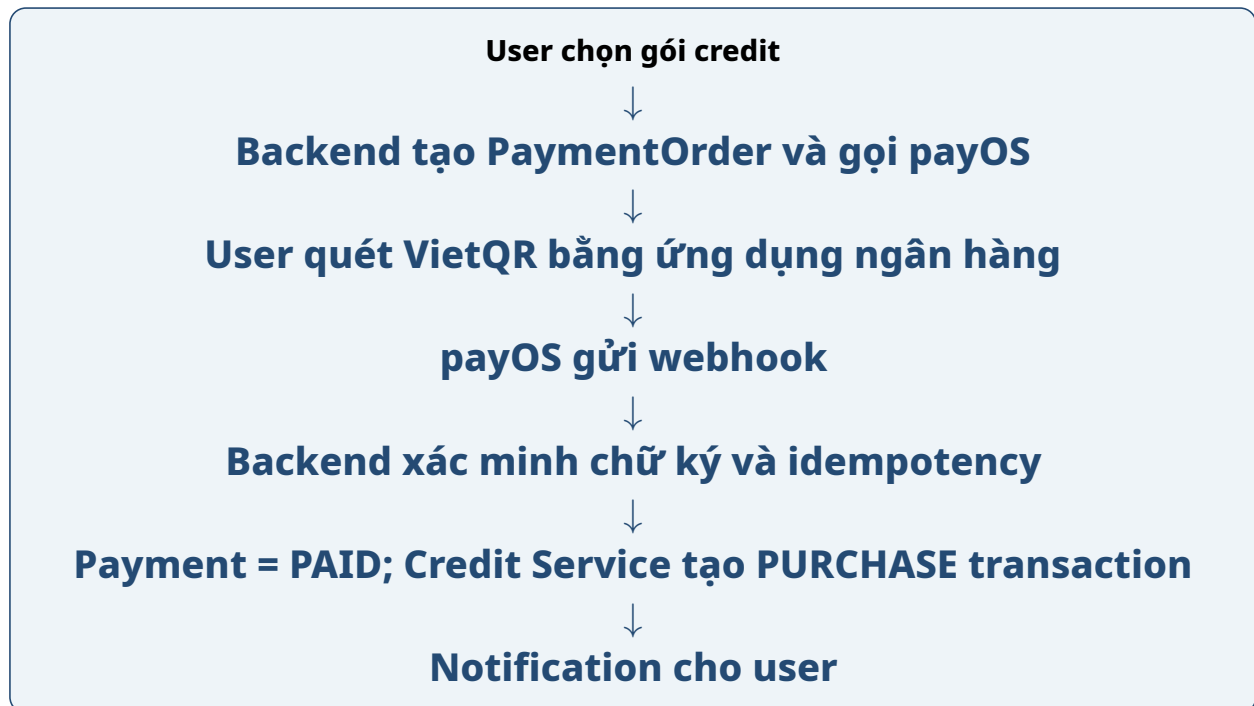
Không tạo ISqlProvider có hàng trăm method. Không bọc mọi framework class chỉ để “có interface”. Abstraction đặt tại biên dịch vụ ngoài hoặc khi cần kiểm thử/đối provider thật sự.

24.3 Implementation baseline

Contract	Implementation
IAiGradingProvider	OpenAiGradingProvider.
IPaymentGateway	PayOsPaymentGateway.
IObjectStorage	LocalObjectStorage ở dev; S3CompatibleObjectStorage ở production.

Contract	Implementation
IEmailSender	Development sink; SMTP/API provider ở production.
IGradingJobQueue	HangfireGradingJobQueue.

25. payOS Payment Flow



25.1 Nguồn sự thật

Không cộng credit từ return URL, screenshot hoặc frontend callback. Webhook đã xác minh là nguồn chính; reconciliation job kiểm tra những payment pending bất thường.

25.2 Payment state

CREATED, PENDING, PAID, EXPIRED, CANCELLED, FAILED, REFUNDED, PARTIALLY_REFUNDED

25.3 Webhook idempotency

Lưu event ID/payload hash, signature verification result, received time, processed time và outcome. Duplicate webhook trả success phù hợp nhưng không grant credit lần hai.

26. Support, grading dispute và refund

Support ticket liên kết trực tiếp Submission, PaymentOrder hoặc CreditTransaction. Admin không sửa wallet; action Re-grade/Refund/Reject gọi service tương ứng và tạo audit. Ticket có messages, attachment, status, assignee, resolution và SLA metric.

26.1 Auto release

Credit tự release khi grading thất bại hoàn toàn hoặc hệ thống không tạo assessment. Khiếu nại chất lượng cần admin review; có thể re-grade miễn phí hoặc hoàn credit.

27. Progress, Mistake Bank và Redo

Giai đoạn đầu query từ assessment, criteria, error occurrence và submission. Khi dữ liệu lớn, recurring job tạo snapshot và aggregate. Insight phải có minimum evidence threshold theo loại thống kê; dữ liệu có đến đâu hiển thị đến đó nhưng không đưa kết luận giả.

Redo tạo submission mới và giữ liên kết với original. Không ghi đè bài cũ hoặc result cũ.

28. Object Storage

Essay và structured grading result lưu database. Object storage dùng cho avatar, support attachment và export. Production ưu tiên storage ngoài VPS; object key ngẫu nhiên, private access, MIME/size validation, signed URL hoặc stream qua backend, lifecycle deletion theo retention.

29. Notification và email

MVP dùng in-app notification lưu database và frontend polling. Email là channel bổ sung, gửi qua background job. Notification delivery failure không rollback grading/payment đã thành công.

30. API Design

30.1 Route convention

```
/api/v1/auth  
/api/v1/profile  
/api/v1/prompts  
/api/v1/drafts  
/api/v1/submissions  
/api/v1/results  
/api/v1/mistakes  
/api/v1/progress  
/api/v1/redo  
/api/v1/credits  
/api/v1/payments  
/api/v1/support  
/api/v1/admin
```

30.2 Problem Details

Mọi lỗi dùng format nhất quán có code, detail, traceId và status HTTP. UI không phụ thuộc exception text nội bộ.

30.3 Pagination và filter

Danh sách lớn dùng page/cursor, page size giới hạn, sort whitelist và filter có validation. Không trả toàn bộ user/submission/payment.

30.4 Idempotency key

Bắt buộc hoặc hỗ trợ cho submit essay, create payment order, refund, credit adjustment và webhook processing.

31. Database entities cấp cao

31.1 Identity

- Users, ExternalLogins, UserSessions.
- Roles, Permissions, UserRoles, RolePermissions.
- UserProfiles.

31.2 Writing và grading

- WritingPrompts, PromptTags, ModelEssays.
- WritingDrafts, WritingSubmissions.
- GradingPromptVersions, GradingAttempts, WritingAssessments.
- AssessmentCriteriaScores, SentenceCorrections, VocabularySuggestions, AiModelCalls.

31.3 Learning

- ErrorTaxonomies, ErrorOccurrences, UserMistakeStats.
- UserProgressSnapshots, RedoRecommendations, RedoAttempts.

31.4 Billing và support

- CreditWallets, CreditTransactions, CreditReservations, CreditPackages.
- PaymentOrders, PaymentWebhookEvents, Refunds.
- SupportTickets, SupportMessages, SupportAttachments.

31.5 Operations

- Notifications, AppSettings, SecurityEvents, BlockedClients.
- RateLimitEvents, AdminAuditLogs, ProductEvents.
- DataDeletionRequests, DeletionLedger.

Chi tiết field, index, FK và migration thuộc 05_Database_Design hoặc phụ lục database riêng.

32. Configuration và secret management

32.1 Environment secret

Database connection, Google Client Secret, OpenAI API key, payOS key/checksum, storage credential và email credential dùng environment variable/secret store. Không lưu trong AppSettings UI và không commit repository.

32.2 Business settings

MaxEssayWords, CreditCostPerGrading, retry count, free credit, budget cap, concurrency, retention days và feature toggle có thể quản lý qua AppSettings có validation/audit.

33. Logging, Audit và Observability

33.1 Structured log fields

TraceId, UserId, SubmissionId, PaymentOrderId, JobId, Module, EventName, ElapsedMs và Status. Không log cookie, token, secret hoặc toàn bộ essay.

33.2 Audit

Audit cho role change, user suspend, session revoke, credit adjustment, refund, prompt publish, taxonomy edit, settings change, AI toggle, maintenance và quyền xem essay support.

33.3 Health check

/health/live kiểm tra process; /health/ready kiểm tra database, Hangfire storage và cấu hình bắt buộc. Không gọi OpenAI/payOS trong mọi health request.

33.4 Metric

API latency/error, queue length, job age, grading success/failure, retry, AI token/cost, payment webhook failure, database/storage/disk, CPU/RAM và support backlog.

34. Rate Limiting và abuse control

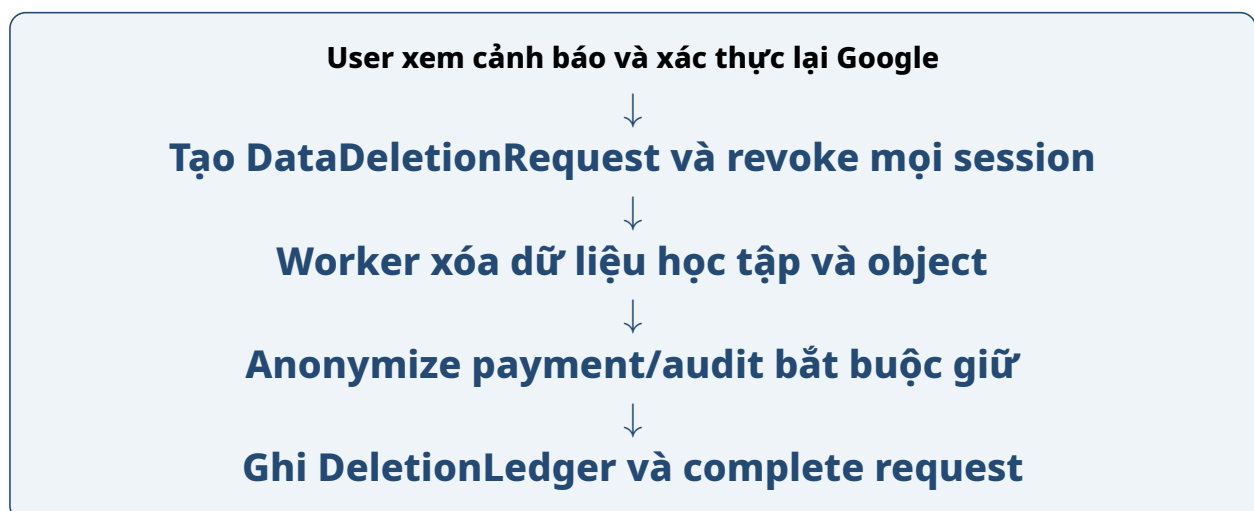
Endpoint/nhóm	Baseline	Key
Google login initiation	Giới hạn hợp lý	IP/device signal.
Autosave	Cao hơn submit	User + draft.
Submit grading	3 lần/5 phút	User.
Submit grading IP	10 lần/5 phút	IP.
Create payment order	Chật hơn API đọc	User.
Support ticket	Chống spam	User.
Admin sensitive action	Rất chặt + re-auth	Admin session.

Rate limit không thay thế credit, budget hoặc permission check.

35. Data Retention

Dữ liệu	Baseline retention
Essay, assessment, mistakes, progress	Trong thời gian tài khoản tồn tại; xóa khi account deletion.
Draft bỏ dở	90 ngày.
Raw AI response	Không lưu hoặc tối đa 7 ngày khi debug có kiểm soát.
Application log	30 ngày.
Error log	90 ngày.
Security event	12 tháng.
Admin audit	24 tháng; payment-sensitive có thể theo payment retention.
Support ticket	12 tháng sau khi đóng.
Support attachment	90 ngày sau khi đóng.
Raw analytics	13 tháng; aggregate ẩn danh có thể lâu hơn.
Database backup	Rolling 30 ngày.
Payment/accounting record	Theo nghĩa vụ pháp luật; baseline thận trọng 10 năm.

36. Account Deletion



Xóa profile, Google identifier, draft, essay, assessment, mistakes, progress, redo, notification, attachment và analytics định danh. Payment/accounting tối thiểu được giữ nhưng tách khỏi identity và không dùng cho marketing. Backup hết hạn theo lifecycle; sau restore phải chạy lại DeletionLedger.

37. Backup và Disaster Recovery

37.1 Backup

- PostgreSQL backup hằng ngày, mã hóa, lưu ngoài VPS.
- Rolling 30 ngày ở baseline.
- Kiểm thử restore định kỳ, không chỉ kiểm tra file tồn tại.
- Object storage có lifecycle/versioning phù hợp.

37.2 Mục tiêu tham khảo

RPO tối đa 24 giờ và RTO vài giờ cho MVP. Khi có nhiều user trả phí, giảm RPO bằng backup thường xuyên hoặc WAL archiving.

38. Failure Scenarios

Sự cố	Xử lý
OpenAI timeout/429/5xx	Retry có giới hạn; circuit breaker; không mất credit.
AI output sai	Corrective retry; fail an toàn nếu vẫn invalid.
Worker restart	Hangfire persist job; idempotent job chạy lại.
API restart	Cookie vẫn hợp lệ nếu Data Protection keys persist.
Enqueue thất bại	Recovery/outbox tìm QUEUED chưa enqueue.
Webhook trùng	Idempotency; không cộng credit trùng.
Webhook không tới	Reconciliation job và nút kiểm tra payment.
Database down	Readiness fail; không nhận mutation mới; cảnh báo admin.
Disk VPS đầy	Alert, log rotation, chặn upload, cleanup.
Double-click submit	Idempotency key; một submission và một reservation.
Restore backup cũ	Chạy migration, DeletionLedger và reconciliation.

39. Security Threat Model cơ bản

Threat	Mitigation
Session hijacking	HTTPS, Secure/HttpOnly, short admin session, revoke, re-auth.
CSRF	Anti-forgery, SameSite, same-origin.
XSS	Encode/sanitize, CSP, không render raw AI HTML.
IDOR	Ownership check và permission ở backend.
Privilege escalation	Policy, role management only by Super Admin, audit.

Threat	Mitigation
Prompt injection	Essay là untrusted content; AI chỉ trả grading schema; không có tool quyền cao.
AI cost abuse	Credit, rate limit, queue, concurrency, budget cap, kill switch.
Fake/replay webhook	Signature verification, event idempotency, timestamp/order check.
Credit race condition	Transaction, row/concurrency locking, unique reservation.
SQL injection	Parameterized EF queries; raw SQL được review.
Malicious attachment	MIME/size allowlist, private storage, no execution.
Secret leakage	Environment secrets, redaction, restricted admin settings.

40. Attachment Security

MVP chỉ cho PNG, JPG, PDF với giới hạn dung lượng. File name user không được dùng làm path; object key ngẫu nhiên. Kiểm tra MIME và extension, không thực thi, private access, signed URL ngắn hạn hoặc stream qua backend, tự xóa theo retention.

41. Scalability Roadmap

41.1 Giai đoạn 1

Một VPS chạy web, API, worker, PostgreSQL và Caddy; external object storage. AI concurrency 3.

41.2 Giai đoạn 2

Tách worker sang VPS riêng, tăng worker instances, managed PostgreSQL, Redis nếu cần distributed rate limit/cache.

41.3 Giai đoạn 3

Nhiều API instances sau load balancer, shared Data Protection keys, external storage, observability chuyên dụng. Chưa cần microservice nếu modular monolith vẫn đáp ứng.

42. Hướng mở rộng Mobile App

Web hoàn chỉnh trước. Mobile app sau này dùng chung API, DTO và account. Mobile authentication có thể dùng Authorization Code + PKCE/token flow riêng; không cần ép web hiện tại dùng JWT. Business logic và authorization vẫn ở backend.

43. Testing Boundary

- Unit test domain rule: word limit, credit reservation, retry classification.

- Integration test PostgreSQL transaction, unique constraint và EF migration.
- Integration test Hangfire enqueue, restart, retry và multi-worker.
- Contract test OpenAI structured parser bằng fixture.
- Webhook signature/idempotency test payOS.
- Authorization test cho role/permission/ownership.
- End-to-end test submit-to-result và payment-to-credit.
- Restore test backup và account deletion replay.

44. Migration, Seed và Admin Bootstrap

Seed roles, permissions, role-permission mapping, error taxonomy, app settings và credit package. Không hard-code production admin trong source. Bootstrap bằng Google email từ environment một lần; user đăng nhập, được cấp SUPER_ADMIN, sau đó khóa bootstrap flow. Admin mới chỉ do Super Admin cấp quyền.

45. State Machine cần đặc tả chi tiết

1. User Account.
2. Draft.
3. Submission.
4. Grading Attempt.
5. Credit Reservation.
6. Payment.
7. Support Ticket.
8. Data Deletion Request.
9. Prompt Publication.
10. Model Essay Review.

Mỗi state machine cần actor, transition, guard, side effect, transaction và audit requirement.

46. Sequence Diagram cần bổ sung

1. Google Login.
2. Autosave với version conflict.
3. Submit + reserve credit + enqueue.
4. Grading success.
5. Grading retry/fail/release.
6. Circuit breaker.

7. payOS checkout + webhook.
8. Duplicate webhook.
9. Payment reconciliation.
10. Support + credit refund.
11. Suspend user + revoke sessions.
12. Account deletion.
13. Restore backup + deletion ledger.

47. Architectural Decision Records

ADR	Quyết định
ADR-001	Modular Monolith thay microservice.
ADR-002	Desktop web-first; mobile app sau.
ADR-003	React + TypeScript + Vite.
ADR-004	ASP.NET Core + PostgreSQL/EF Core.
ADR-005	Google OIDC + application cookie cho web.
ADR-006	Role + permission authorization.
ADR-007	Hangfire cho background jobs, worker riêng ở production.
ADR-008	Một model OpenAI chính và structured output.
ADR-009	Credit ledger + reservation.
ADR-010	payOS + verified webhook.
ADR-011	Polling trước SignalR.
ADR-012	VPS + Docker Compose + Caddy.
ADR-013	Production object storage tương thích S3.

48. Rủi ro kỹ thuật và biện pháp

Rủi ro	Mức	Biện pháp
AI grading không ổn định	Cao	Benchmark, prompt version, schema validation, monitoring và dispute flow.
Chi phí AI tăng	Cao	Credit, budget cap, concurrency, model config và cost dashboard.
Hangfire PostgreSQL provider	Trung bình	Integration test, abstraction queue và backup phương án storage.
Payment webhook sai/trùng	Cao	Signature, idempotency và reconciliation.
Credit sai số dư	Cao	Ledger, transaction, unique constraints và reconciliation.

Rủi ro	Mức	Biện pháp
VPS single point of failure	Trung bình	Offsite backup, restore test, external storage và roadmap tách DB/worker.
Admin lạm quyền	Cao	Least privilege, tài khoản riêng, audit và re-auth.
Data deletion không đầy đủ	Cao	Deletion workflow, object cleanup, ledger và backup lifecycle.

49. Implementation Checklist

49.1 P0 - Foundation

- Repository, Docker Compose, PostgreSQL migration.
- Google Login, cookie, persisted Data Protection keys.
- Role/permission/policy và Super Admin bootstrap.
- Module skeleton, error format, logging và health checks.

49.2 P0 - Core writing/grading

- Prompt, draft, autosave versioning, submission state.
- Credit wallet/ledger/reservation.
- Hangfire worker, grading queue và recovery.
- OpenAI structured output, prompt version và usage.
- Result, history, notification.

49.3 P1 - Product differentiation

- Mistake Bank, Progress, Redo, Model Essay.
- payOS payment, webhook, reconciliation.
- Support tickets, admin modules, audit và security events.

49.4 P1 - Production hardening

- Rate limit, circuit breaker, budget cap và kill switch.
- Object storage, backup, restore test và retention jobs.
- Account deletion và privacy documentation.
- Monitoring dashboard và alerting.

50. Tiêu chí chấp nhận System Design

Tài liệu được xem là đủ để bắt đầu implementation khi:

- Các quyết định stack và deployment không còn mâu thuẫn.

- Module ownership và dependency được thống nhất.
- Auth, permission, credit, payment và grading state machine được chốt.
- Hangfire queue, concurrency, retry, idempotency và recovery được chốt.
- Database provider, migration và backup strategy được chốt.
- Third-party interface và secret boundary rõ.
- Retention, account deletion và audit có baseline.
- Các ADR chính được tạo và review.

51. Kết luận

BandUp sẽ bắt đầu bằng một Modular Monolith trên VPS, dùng React/TypeScript, ASP.NET Core, PostgreSQL, Google Login + cookie-session, Hangfire worker, một model OpenAI, credit ledger và payOS. Thiết kế tập trung vào tính đúng đắn của transaction, idempotency, quyền truy cập và data lifecycle hơn là tối ưu phân tán quá sớm.

Hangfire là lựa chọn phù hợp miễn job được xem là có khả năng chạy lại, business side effect idempotent, retry được kiểm soát và worker được tách khỏi API ở production. Khi hệ thống lớn hơn, API, worker, database và storage có thể tách hạ tầng độc lập mà không cần thay đổi nghiệp vụ cốt lõi.

Nguồn tham khảo

- Hangfire Documentation: <https://docs.hangfire.io/>
- Hangfire - Dealing with Exceptions: <https://docs.hangfire.io/en/latest/background-processing/dealing-with-exceptions.html>
- Hangfire - Configuring Job Queues: <https://docs.hangfire.io/en/latest/background-processing/configuring-queues.html>
- Hangfire - Configuring Degree of Parallelism: <https://docs.hangfire.io/en/latest/background-processing/configuring-degree-of-parallelism.html>
- ASP.NET Core - Google external login: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/social/google-logins>
- ASP.NET Core - Cookie authentication: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/cookie>
- Npgsql EF Core Provider: <https://www.npgsql.org/efcore/>
- payOS Documentation: <https://payos.vn/docs/>
- payOS Webhook and Signature: <https://payos.vn/docs/du-lieu-tra-ve/webhook/>
- OpenAI API Documentation: <https://platform.openai.com/docs/>
- Docker Compose Documentation: <https://docs.docker.com/compose/>

Các nguồn tham khảo kỹ thuật có thể thay đổi theo phiên bản. Khi triển khai, đội phát triển phải khóa version package, đọc release notes và kiểm thử migration/integration trước production.